

PRÁCTICA 4 de PSP (3/11/2025):

MVC final - Bases de datos

Fecha tope entrega	13 de noviembre.	Fecha tope defensa	20 de noviembre.
Tipo	Grupo de 2 o 3 personas	Entrega	Github

Requisitos:

- 1.- Desarrolla una aplicación sobre el esquema de base de datos: Cliente → Cuenta.
 - 1.1.- Se recomienda el sistema de base de datos Apache Derby integrado en el IDE Netbeans 8.2.
 - 1.2.- Tipos de datos obligatorios: int, float, GregorianCalendar, String.
 - 1.3.- Cliente tendrá los campos usuario y contraseña.
 - 1.4.- Habrá una imagen la tabla cliente.
 - 1.5.- Existirán PK y FK. Estás últimas con el formato "cliNumero": "cli" son las 3 primeras letras de la tabla donde es PK ("Cliente") y "número" del nombre del campo en dicha tabla.
 - 1.6.- (Voluntario) En la clase Cliente tendrá el **campo calculado** "saldo total" que se obtiene de la suma del campo "saldo" de todas sus cuentas.
- 2.- Basándote en una de las tres variantes del MVC desarrolla una aplicación en Java. Habrá cuatro paquetes: modelo, vista, controlador y app.
 - Vista-
- 3.- Se basará en clases con un JFrame y varios JPanel.
- 4.- Menú contendrá las opciones: **Validar** (entrar y salir), **ver** y **resumen**.
 - 4.1.- Los elementos del menú estarán activados o desactivados según la lógica de la aplicación.
- 5.- Funcionamiento: tras elegir en el menú "Validar / log in" se pedirá un nombre del Cliente y su contraseña.
 - 5.1.- Si la validación ha tenido éxito, activará el resto de menús.
 - 5.2.- La conexión a la base de datos, con la cuenta de administrador de la BDs es transparente al usuario y será automática.
- 6.- En el panel "**ver**":
 - 6.1.- Mostrará uno a uno los registros de la tabla Cuenta asociados al elemento de la tabla Cliente validado.
 - 6.2.- (Voluntario) También se podrá modificar el campo para los cálculos (cuenta.saldo).
 - 6.3.- Se podrá **avanzar** y **retroceder** entre las cuentas directamente sobre la BDs.
- 7.- Panel "**resumen**":
 - 7.1.- Se mostrarán todos los datos de la tabla Cliente, incluida la **imagen**.
 - 7.2.- En un JTable o JList se mostraran los campos cuenta.numero y cuenta.saldo.
 - 7.3.- (Voluntario) Tendrá un botón "Calcular": tras pulsarse mostrará el cálculo obtenido en una ventana emergente y después actualizará el valor **real del campo calculado** según la BDs. En el ejemplo: el saldoTotal del cliente visualizado, inicialmente tendrá el valor guardado en la BDs., que no tiene por que ser el correcto.
- 9.- Aparecerán **ventanas informativas prediseñadas** (JOptionPane).
- Modelo -
- 10.- Crear **una clase por tabla** de la BDs. con sus métodos get y set, pero **no** los métodos para guardar y modificar en la BDs.
- 11.- Gestión BDs.: crear 4 clases que gestionen las operaciones sobre la base de datos (antes en el controlador, ahora en el modelo):
 - 11.1.- Una orientada a la parte común: abrir, cerrar y obtener (get) la conexión. Es la única que puede ser estática.
 - 11.2.- Clase usada por el JPanel "ver". A partir de la conexión crea un Statement (SCROLL y UPDATABLE) y un ResultSet. Sus métodos permitirán:
 - Avanzar, retroceder, ir a inicio, ir a fin, etc. sobre el ResultSet.
 - Guardar y/o modificar un registro cliente (insertRow o updateDate).
 - Crear y devolverán el objeto de la clase Cuenta donde esté el ResultSet (getActual).
 - 11.3.- Clase pensada para el JPanel "resumen". Se encargará de rellenar el JTable o Jlist con Statement y ResultSet simples. Sus métodos se encargarán de:
 - Devolver una **collection** con objetos de Cuenta.
 - 11.4.- La cuarta clase se usará también en panel "resumen", pero para la tabla Cliente: Statement y ResultSet simples. Sus métodos se encargarán:
 - Obtener, Intercambiar y actualizar objetos de Cliente.
- 12.- Se pueden usar las tres formas del MVC:
 - MVC inicial: es la dada hasta ahora pero el controlador se integra en el modelo en modo de herramientas.
 - MVC práctico: existe el controlador y la vista asume parte de su funcionalidad.

- MVC puro: el controlador asume todo el control.

13.- Se puede utilizar otro sistema de base de datos con Mysql, MariaDB, Postgresql, etc.

- Se aconseja dockers.

14.- Se recomienda un sistema de errores centralizado.